Name: Nguyễn Dương Phúc
Student ID: 24521386
Class: CS4323.Q12.CTTT

OS
LAB 3

CHECKLIST

1. Homework

	1	2	3
Explanation	☒	☒	☒
Your code	☒	☒	☒
Results and discussions	☒	☒	☒

Mark: 10/10 (self-scrores)

**Notice: Export PDF file, name format:*
<MSSV>_LAB3.pdf

2. Lab 3

1. Given the file gradebook.csv which contain grades of a class. Research how to use IFS to read csv file. Use shell script to:

- **Read Student ID from command line**
- **Use Student ID to find and print name, original grade, and letter-based grade of corresponding student**

Answer:

Step 1: Create a new shell script file

```
yangpuck@NguyenDuongPhuc:~$ nano find_grade.sh
```

Step 2: Write the script with comments explaining each step:

- Prompt the user for Student ID

```
Open ▾                    find_grade.sh
                                ~/
#!/bin/bash

read -p "Enter Student ID: " target_id
target_id=$(echo "$target_id" | xargs)  # remove khoảng trắng
```

- Check if the CSV file exists

```
if [ ! -f gradebook.csv ]; then
    echo "File gradebook.csv not found!"
    exit 1
fi
```

- Read the CSV file line by line using a loop

```
while IFS=';' read -r id name grade
do
    # Loại CRLF và khoảng trắng thừa
    id=$(echo "$id" | tr -d '\r' | xargs)
    name=$(echo "$name" | tr -d '\r' | xargs)
    grade=$(echo "$grade" | tr -d '\r')

    # BỎ qua dòng header
    if [ "$id" == "StudentID" ]; then
        continue
    fi
```

- Compare the input ID with the current line

```
        if [ "$id" == "$target_id" ]; then
            # Chuyển grade thành letter grade
            if [ "$grade" -ge 900 ]; then letter="A"
            elif [ "$grade" -ge 800 ]; then letter="B"
            elif [ "$grade" -ge 700 ]; then letter="C"
            elif [ "$grade" -ge 600 ]; then letter="D"
            else letter="F"
            fi

            echo "Name: $name"
            echo "Original Grade: $grade"
            echo "Letter Grade: $letter"
            exit 0
        fi
    done < gradebook.csv

    echo "Student ID not found!"
```

Notes / Explanation:

- IFS=';' is used to split each line into columns.
- tr -d '\r' removes Windows carriage return characters (^M).
- The script skips the header line, then compares the input ID with each line.

- If a match is found, it prints the student's name, original numeric grade, and letter grade.
- If no match is found, it prints "Student ID not found!"

Step 3: Give execution permission to the script

```
yangpuck@NguyenDuongPhuc:~$ chmod +x find_grade.sh
```

Step 4: Run the script

```
yangpuck@NguyenDuongPhuc:~$ ./find_grade.sh
Enter Student ID: 23560181
Name: Student 67
Original Grade: 384
Letter Grade: F
```

2. Write a shell script to let user input their age, then:

- **Check if input is a number and its value is between 0 and 100.**
- **If input is less than 12 → Print "You are a child"**
- **If input is from 12 – 18 → Print "You are a teenager"**
- **If input is greater than 18 → Print "You are an adult"**

Answer:

Step 1: Create a new shell script file

```
yangpuck@NguyenDuongPhuc:~$ nano check_age.sh
```

Step 2: Write the script with comments explaining each step:

- Prompt the user to enter their age

```
  GNU nano 2.9.3                    check_age.sh

#!/bin/bash

# Step 1: Prompt the user to enter their age
read -p "Enter your age: " age
```

- Check if the input is a valid number
    - Use regex to ensure only digits

```
# Step 2: Check if the input is a number between 0 and 100
if ! [[ "$age" =~ ^[0-9]+$ ]]; then
    echo "Invalid input! Please enter a number."
    exit 1
fi
```

- Check if the age is between **0** and **100**

```
if [ "$age" -lt 0 ] || [ "$age" -gt 100 ]; then
    echo "Invalid age! Please enter a number between 0 and 100."
    exit 1
fi
```

- Determine age category:

  Age less than **12** → child

  Age **between 12** and **18** → teenager

  Age greater **than 18** → adult

```
# Step 3: Determine age category
if [ "$age" -lt 12 ]; then
    echo "You are a child"
elif [ "$age" -le 18 ]; then
    echo "You are a teenager"
else
    echo "You are an adult"
fi
```

Step 3: Give execution permission

```
yangpuck@NguyenDuongPhuc:~$ chmod +x check_age.sh
```

Step 4: Run the script (test case)

```
yangpuck@NguyenDuongPhuc:~$ ./check_age.sh
Enter your age: 8
You are a child
yangpuck@NguyenDuongPhuc:~$ ./check_age.sh
Enter your age: 18
You are a teenager
yangpuck@NguyenDuongPhuc:~$ ./check_age.sh
Enter your age: 67
You are an adult
yangpuck@NguyenDuongPhuc:~$ ./check_age.sh
Enter your age: 696
Invalid age! Please enter a number between 0 and 100.
yangpuck@NguyenDuongPhuc:~$ ./check_age.sh
Enter your age: UIT
Invalid input! Please enter a number.
```

3. Write a program that read 02 arguments from command line: first argument is a string while second argument is a path.

Check if that string exists in any text file (for example, test.txt) in the provided path. Print the line containing the string.

Answer:

Step 1: Create the script file

```
yangpuck@NguyenDuongPhuc:~$ nano find_string_files.sh
```

Step 2: Write the script with comments explaining each step:

- Check if two arguments are provided: The script first checks if exactly two arguments (a string and a directory path) are provided. If not, it prints a usage message showing the correct format and exits.

```
  GNU nano 2.9.3                    find_string_files.sh

#!/bin/bash

# Check if two arguments are provided
if [ "$#" -ne 2 ]; then
    echo "Usage: $0 <string> <directory>"
    exit 1
fi
```

- Assign arguments to variables:
  - The first argument ($1) is assigned to 'search_string' (the string to search for).
  - The second argument ($2) is assigned to 'search_dir' (the directory to search in).

```
# Assign arguments to variables
search_string=$1
search_dir=$2
```

- Check if the directory exists: The script verifies whether the provided directory exists using '-d'. If the directory does not exist, it prints an error message and exits.

```
# Check if the directory exists
if [ ! -d "$search_dir" ]; then
    echo "Error: Directory $search_dir does not exist."
    exit 1
fi
```

- Search for the string in text files: 'find' lists all .txt files in the given directory: For each file, the script uses 'grep -q' to check silently if the string exists. If the string is found, it prints the file name and the line(s) containing the string. The 'found' variable is set to 1 to indicate at least one match was found.

```
# Find and search text files for the string
found=0
while read -r file; do
    if grep -q "$search_string" "$file"; then
        echo "Found in file: $file"
        grep "$search_string" "$file"
        found=1
    fi
done < <(find "$search_dir" -type f -name "*.txt")
```

- Handle the case where no matches are found: After checking all .txt files, if 'found' is still 0, the script prints a message indicating that the search string was not found in any file in the directory.

```
# If no matches found
if [ $found -eq 0 ]; then
    echo "No matches found for '$search_string' in $search_dir."
fi
```

Step 3: Give execution permission

```
yangpuck@NguyenDuongPhuc:~$ chmod +x find_string_files.sh
```

Step 4: Prepare test data

```
yangpuck@NguyenDuongPhuc:~$ mkdir ~Documents
yangpuck@NguyenDuongPhuc:~$ echo "This is an example line" > ~/Documents/file1.
txt
yangpuck@NguyenDuongPhuc:~$ echo "Another line without the term" > ~/Documents/
file2.txt
yangpuck@NguyenDuongPhuc:~$ echo "The example is here again" > ~/Documents/file
3.txt
yangpuck@NguyenDuongPhuc:~$ echo "Nothing related here" > ~/Documents/file4.txt
```

Step 5: Run the script

```
yangpuck@NguyenDuongPhuc:~$ ./find_string_files.sh "example" ~/Documents
Found in file: /home/yangpuck/Documents/file3.txt
The example is here again
Found in file: /home/yangpuck/Documents/file1.txt
This is an example line
yangpuck@NguyenDuongPhuc:~$ ./find_string_files.sh "UIT" ~/Documents
No matches found for 'UIT' in /home/yangpuck/Documents.
```